

Assignment-Week 01 – Day 01 :

1) Find 10 different model providers and their models released in last 12 months

1. OpenAI

- GPT-5 (Aug 2025), GPT-5.1 (Nov 2025), GPT-5.6 line — Sol, Luna, Terra (Jul 2026)
- ChatGPT Work agent, powered by GPT-5.6 (Jul 2026)

2. Anthropic

- Claude Opus 4.1, 4.5, 4.8
- Claude Sonnet 4, 4.5, and 5
- Claude Fable 5 and Claude Mythos 5 — first Mythos-tier models (Jun 2026), with Mythos 5 limited to approved organizations
- Claude Opus 5 (Jul 24, 2026) — the newest flagship, priced roughly half of Fable 5

3. Google DeepMind

- Gemini 3 family with Deep Think mode
- Gemini 3.6 Flash (Jul 21, 2026) — newer, more efficient workhorse model
- Overhauled image/video generation stack, including in-context image editing

4. Meta

- Llama updates and the newer "Muse" line — Muse Spark 1.1 (Jul 9, 2026)

5. xAI

- Grok 4.20 (Feb 2026) — parallel multi-agent architecture (Grok, Harper, Benjamin, Lucas)
- Grok 4.3 (Apr 30, 2026) — new flagship, most permissive guardrails, native real-time X data
- Grok 4.5 mentioned as a further update

6. DeepSeek

- DeepSeek-V4 (preview, Apr 2026 → GA Jul 20, 2026) — stronger agentic, math, and code capabilities
- DeepSeek-V4-Flash-0731 (Jul 31, 2026)

7. Mistral AI

- Mistral Large 3 plus compact dense models, released under Apache 2.0 open-weights license

8. Alibaba (Qwen)

- Qwen3.7 Flash (Jul 27, 2026)

9. Moonshot AI

- Kimi K3 (Jul 16, 2026)

10. Z.ai / Cursor (also actively tracked as frontier-adjacent providers)

- Cursor and Z.ai have shipped their own model updates through 2026, though details are less consolidated in trackers than the above.

Details to collect:

Model Provider	Model	Release Date	Cut off Date	Cost of tokens	Context Window	Type	When to use this model
----------------	-------	--------------	--------------	----------------	----------------	------	------------------------

AI Model Providers - Last 12 Months Release Comparison

Model Provider	Model	Release Date	Cutoff Date	Cost (per 1M tokens, in/out)	Context Window	Type	When to Use
OpenAI	GPT-5.5	May 2026	Dec 1, 2025	\$5 / \$30	1.05M tokens	Multimodal, reasoning (adjustable reasoning effort)	Complex professional work — deep reasoning, long documents, high-stakes coding/analysis
Anthropic	Claude Opus 4.5	Nov 24, 2025	Mar 2025	\$5 / \$25	1M tokens (beta)	Multimodal, agentic, computer-use	Coding, multi-step agents, tasks needing top-tier reliability at lower cost than older Opus
Google DeepMind	Gemini 3 Pro	Nov 18, 2025	Jan 2025	\$2 / \$12	1M tokens	Multimodal (text/image/video/audio), agentic (Deep Think mode)	Complex multimodal reasoning, agentic workflows via Google Antigravity
Meta	Muse Spark	Apr 8, 2026	Not publicly disclosed	Not publicly disclosed (closed, API-only)	Not publicly disclosed	Closed-weight reasoning model (Meta's first proprietary frontier release)	Powers the Meta AI app/website now; use where Meta's assistant ecosystem is the target surface
xAI	Grok 4.3	Apr 30, 2026	Dec 2025	\$1.25 / \$2.50	1M tokens	Multimodal, real-time X data integration	Everyday reasoning + fresh/real-time info, document/video generation
DeepSeek	DeepSeek V4 (Pro)	Apr 24, 2026 (preview) to GA ~Jul 2026	Not clearly disclosed	\$0.435 / \$0.87	1M tokens	Open-weight (MIT), agentic/coding/math reasoning	Budget-conscious teams needing frontier-adjacent reasoning + self-hosting option
Mistral AI	Mistral Large 3	Dec 2, 2025	Not publicly disclosed	\$0.50 / \$1.50	256K tokens	Open-weight (Apache 2.0), multimodal MoE	EU-jurisdiction needs, open-weight self-hosting, long-document/multilingual work at low cost
Alibaba	Qwen3.8 Max	Aug 3, 2026	Not publicly disclosed	\$2 / \$6	1M tokens	Multimodal MoE (2.4T params, open weights pending)	Large-scale multimodal agentic work wanting an open-weight frontier option
Moonshot AI	Kimi K3	Jul 16, 2026	Not publicly disclosed	Not yet confirmed in trackers	1M tokens	Open-weight, agentic	Strong open alternative where Western flagship models are restricted/unavailable
Z.ai (Zhipu AI)	GLM-5.2	Jun 13, 2026	Not publicly disclosed	~\$0.70-1.40 / \$2.20-4.40 (varies by provider)	1M tokens	Open-weight (MIT), long-horizon coding agent	Repository-scale coding, long multi-step engineering tasks at low cost

2) Read 2–3 articles about how transformer works?**Goal to understand:****(a) Self-attention mechanism****(b) Vector embedding**

(c) Feed forward Neural Network

Recommended reads (2-3)

1. **Transformer Explainer (visual, interactive)** — poloclub.github.io/transformer-explainer/
Best starting point. It's a live, visual walkthrough built on GPT-2 that lets you watch how transformer models operate on the principle of next-token prediction, using self-attention to process entire sequences and capture long-range dependencies. Great for building intuition before reading math-heavy stuff. [Polo Club of Data Science](#)
2. **"Understanding and Coding the Self-Attention Mechanism" by Sebastian Raschka** — sebastianraschka.com/blog/2023/self-attention-from-scratch.html
The gold standard explainer. It walks through self-attention as the mechanism that lets a model weigh the importance of different elements in an input sequence and dynamically adjust their influence on the output, then extends into multi-head and cross-attention. Has Python code — good since you're technical (SDET background). [Sebastian Raschka](#)
3. **"What is self-attention?" — IBM** — ibm.com/think/topics/self-attention
Shorter, cleaner conceptual piece if you want something less code-heavy first. Frames self-attention as weighing the importance of tokens in an input sequence to better understand the relations between them. [IBM](#)

How Transformers Work (Big Picture)

What is a Transformer?

A **Transformer** is a deep learning architecture introduced in 2017 in the paper:

"Attention Is All You Need"

Unlike older models (RNN/LSTM), Transformers can process **all words in a sentence simultaneously**, making them much faster and better at understanding long-range relationships.

Overall Flow

Input Sentence



Tokenization



Vector Embeddings



Positional Encoding



Self-Attention



Feed Forward Neural Network



Output Representation



Prediction (Next Word / Translation / Classification)

Every Transformer block mainly contains:

1. Self-Attention
2. Feed Forward Neural Network

These blocks repeat many times (12, 24, 48...).

Example

Sentence:

The animal didn't cross the street because it was tired.

Question:

What does "**it**" refer to?

Transformer understands

it → animal

instead of

it → street

using **Self-Attention**.

Article 2: Self-Attention Mechanism

This is the **heart of the Transformer**.

Without Self-Attention,
the model only sees nearby words.

With Self-Attention,
every word can look at every other word.

Example

Sentence

The cat sat on the mat because it was soft.

When reading

it

the model asks

Which previous word is most related to me?

Possible attention scores

Word	Attention
The	0.01
cat	0.15
sat	0.02
mat	0.72
because	0.04

Word	Attention
soft	0.06

Highest score:

mat

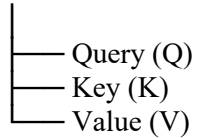
Therefore

it = mat

How does Self-Attention work?

Every word is converted into **three vectors**:

Word



Think of them like this:

Query

"I'm looking for related information."

Key

"I contain this type of information."

Value

"This is the actual information."

Everyday Analogy

Imagine searching YouTube.

You type:

Transformer Tutorial

Your search text

↓

Query

Every video has keywords

↓

Key

The actual video

↓

Value

Search engine compares

Query vs Keys

Then returns the Values of the best matches.

Exactly what Self-Attention does.

Step-by-step

Sentence:

I love AI

Word vectors

I

love

AI

For every word:

Generate

Query

Key

Value

Then compute similarity

Query $\times$ Key

Higher similarity

↓

Higher Attention Score

Scores become probabilities

↓

Multiply by Value

↓

Combine

↓

New word representation

Example

AI

looks at

I

love

AI

Suppose

I 10%

love 35%

AI 55%

Final embedding becomes

0.10(I)

+

0.35(love)

+

0.55(AI)

This is why the model understands context.

Why is Self-Attention powerful?

Instead of only reading left to right,
every word can communicate with every other word.

Word1 $\longleftrightarrow$ Word2

↑ ↓

Word3 $\longleftrightarrow$ Word4

Everything is connected.

Article 3: Vector Embeddings

Computers cannot understand text.

They only understand numbers.

Step 1

Sentence

I love AI

Initially

I

love

AI

are just strings.

Need conversion into numbers.

Bad Approach

Assign IDs

I = 1

love = 2

AI = 3

Problem:

The model thinks

$3 > 2 > 1$

No meaning.

Better Approach

Represent each word using hundreds or thousands of numbers.

Example

AI

[

0.32,

-0.51,

0.91,

...
768 values
]

This is called an **Embedding Vector**.

Why?

Words with similar meanings have similar vectors.

Example

King

[0.91
0.32
...]

Queen

[0.88
0.35
...]

Very close.

Meanwhile

Car

[-0.62
0.12
...]

Far away.

Famous Example

King

-

Man

+

Woman

≈

Queen

The vectors capture semantic relationships.

Why 768 numbers?

Models use different dimensions.

Model	Embedding Size
BERT Base	768
GPT-2 Small	768
GPT-3	12288
Llama 3 (varies)	Different sizes

More dimensions allow the model to encode richer information, though the exact size depends on the architecture.

Embedding Analogy

Imagine every word is placed on a huge map.

Dog ----- Puppy

Car

Bus

Vehicle

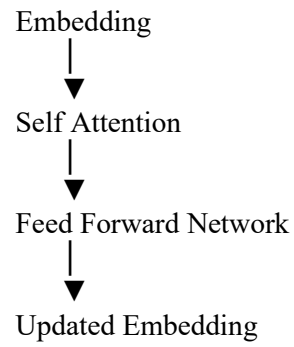
Similar meanings stay close together.

Article 4: Feed Forward Neural Network (FFN)

After Self-Attention,
every word has gathered information.

Now,
the model processes each word independently through the same neural network.

Pipeline



What does FFN do?

Think of Self-Attention as:
Gathering information.

Think of FFN as:
Processing and refining that information.

Structure

Input



Linear Layer



Activation Function (ReLU or GELU)



Linear Layer



Output
Simple diagram
768



3072

↓

Activation

↓

768

The network first expands the representation, applies a non-linear transformation, then compresses it back to the original size.

Why expand?

More neurons allow the model to learn more complex patterns.

Imagine

Small Room

↓

Large Factory

↓

Finished Product

The larger hidden layer gives the model more capacity to transform the information before returning it to the original embedding size.

Everyday Analogy

Self-Attention:

"Collect information from everyone."

FFN:

"Think deeply about that information yourself."

How All Three Work Together:

Sentence

↓

Tokenization

↓

Embeddings

↓

Positional Encoding

↓

Self Attention
(collect context)

↓

Feed Forward Neural Network
(process context)

↓

Repeat many Transformer blocks

↓

Prediction

↓

Next Word

Real Example

Input:

The dog chased the ball because it was fast.

Step 1 – Embeddings:

Convert every word into vectors.

Dog → Vector

Ball → Vector

Fast → Vector

Step 2 – Self-Attention

The word

it

looks at

dog

ball

fast

and determines which words are most relevant.

Step 3 – FFN

Each contextualized word vector is transformed to extract higher-level features and prepare it for the next Transformer layer.

Step 4

Repeat this process through many Transformer layers.

Step 5

Predict the next word or generate the final output.

Quick Revision Table:

Concept	Purpose	Simple Analogy
Vector Embedding	Converts words into meaningful numerical vectors	GPS coordinates for words
Self-Attention	Lets each word focus on the most relevant other words	Everyone in a meeting listening to the right person
Feed Forward Neural Network	Refines each contextualized word representation	A personal expert analyzing gathered information
Transformer Block	Self-Attention + FFN	One complete processing stage
Stack of Transformer Blocks	Repeated refinement of representations	Multiple rounds of discussion and review